

Design Model:

State Space (S):

$$s = (p_1, p_2, p_3, \dots, p_k)$$

Where:

k : the number of drones in the centralized multi-drones system.

p_i : $p_i = (x_i, y_i, z_i) \in \mathbb{Z}^3$ is the 3D grid cell position of the drone i .

Each coordinate: $0 \leq x_i < n_x, 0 \leq y_i < n_y, 0 \leq z_i < n_z, x_i \in \mathbb{Z}, y_i \in \mathbb{Z}, z_i \in \mathbb{Z}$

$$|S| = (n_x n_y n_z)^k$$

Action Space (A):

$$a = (a_1, a_2, a_3, \dots, a_k) \in A_1 \times A_2 \times A_3 \times \dots \times A_k$$

Where:

k : the number of drones.

a_i : Action for the drone i . $a_i \in A_i$,

$$A_i = \{(\Delta x, \Delta y, \Delta z) \mid \Delta x, \Delta y, \Delta z \in \{-1, 0, 1\}\} \setminus \{(0, 0, 0)\}$$

$$|A_i| = 26$$

$$|A| = |A_i|^k = 26^k$$

Transition Function (T):

Assume each drone is independent. For each drone i , we know its transition function is T_{known} .

$$s = (p_1, p_2, p_3, \dots, p_k)$$

$$a = (a_1, a_2, a_3, \dots, a_k)$$

$$s' = (p_1', p_2', p_3', \dots, p_k')$$

For the state s , there are 3 situations: Goal, Normal, Collision.

$$T(s, a, s') = \begin{cases} \prod_{i=1}^k T_{known}(p_i, a_i, p_i') & s = Normal, s' \in \{Normal, Goal\} \\ 1 & s, s' \in \{Goal, Collision\}, s = s' \\ 0 & other \end{cases}$$

It means when s is Normal case and s' has no collision, we multiple all T_{known} of each drone. If s and s' are same and both all Goal or Collision, the probability is 1. Other case is 0.

Reward Function (R):

$\forall a \in A, s \in S$, The reward function $R = R_{terminate} + R_{cost}$

$$\text{If the } s' \text{ is goal: } R_{terminate}(s, a, s'_{goal}) = 100$$

$$\text{If the } s' \text{ exists collision: } R_{terminate}(s, a, s'_{collision}) = -50$$

$$\text{For each step: } R_{cost}(s, a, s') = -1$$

Pseudo Code:

```
Class StateNode:
  Initialize:
    parent, children,          # node
    expanded_actions, N_s, Q_s_a_s  # edge (1D array)
    children                  # children (1D array)
    from_action_index         # index from parent's children
    expanded_reward
    done
    untried_actions

Class MCTTree:
  Initialize:
    root_node, env, EXPLORATION_COE, MAX_SIMULATION_STEP

Function SELECTION ():
  node ← root

  loop do:
    # case 1: leaf node
    if node.children empty:      return node

    # case 2: not fully expanded
    if untried_actions not empty: return node

    # case 3: fully expanded
    for each expanded action with index i:
      ucb[i] ← Q(s, a_i) + c * sqrt(ln(N(s))/N(s, a_i))

    best_action_idx ← index(MAX(ucb))

    # update
    node ← node.children[best_action_idx]

Function EXPANSION(select_node):
  # Assume the select_node is not done state

  action ← RANDOM_REMOVE_UNTRIED_ACTIONS()
  done ← SIMULATE(select_node.state, action)

  if done:
    try_actions ← []          # done do not need to expand
  else:
    try_actions ← GENERATE_TRY_EXPANSION_ACTIONS(state, max_b)

  expansion_node ← new StateNode
  select_node.ADD_CHILD(expansion_node, action)

  return expansion_node
```

```

Function SIMULATION(expanded_node):
    # Assume the expanded_node is not done state

    # initial state
    state ← expanded_node.state
    done ← expanded_node.done
    step_count, total_reward ← 0, 0

    loop do:
        if reach done or step_count == MAX_SIMULATION_STEP:
            break loop
        sim_action ← HEURISTIC_SIMULATION_ACTION(state)

        next_state, reward, next_done ← SIMULATE(state, sim_action)
        total_reward ← total_reward + reward *  $\gamma^{\text{step\_count}}$ 

        # update
        state, done ← next_state, next_done
        step_count ← step_count + 1

Function BACK_PROPAGTION(start_node, reward):
    # Initialize
    node ← start_node
    node.N_s ← node.N_s + 1

    loop do:
        parent ← node.parent
        if parent == None:
            break loop:

        idx ← node.from_action_index
        n_s_a ← parent.n_s_a_s[idx]
        n_s_a ← n_s_a + 1
        parent.n_s_a_s[idx] ← n_s_a

        Q_s_a_old ← parent.Q_s_a_s[idx]
        parent.Q_s_a_s[idx] ← Q_s_a_old + (reward - Q_s_a_old) / n_s_a

        parent.N_s ← parent.N_s + 1

        # update
        node ← parent

Function GENERATE_TRY_EXPANSION_ACTIONS(state, max_branch)
    # Create the actions that should choose in expansion stage

    action_vector ← HEURISTIC_ORDERED_VALID_ACTIONS(state)
    drone_actions_tuples ← [tuple each(drone_id, actions)]

    k_a_record ← []
    remain_actions ← action_vector
    remain_branch ← max_branch
    top_k ← 0

```

```

loop do:
  if left is empty:      break loop
  top_k  $\leftarrow \frac{\text{num}(\text{remain\_actions})}{\sqrt{\text{remain\_branch}}}$ 

  less_equ_k_actions, upper_k_actions  $\leftarrow$  [], []

  for each actions in remain_actions:
    if num(actions)  $\leq$  top_k: add in less_equ_k_actions
    else:                    add in upper_k_actions

  # All remain actions at least can set top k
  if less_equ_k_actions is empty:      break loop

  # deal with leq k actions, choose all element
  For each actions in less_equ_k_actions:
    drone_id  $\leftarrow$  drone_actions_tuples(use actions get id)
    k_a_record.add(tuple(drone_id, num(actions)))

  fix_branch  $\leftarrow$  |Aleq_1| + |Aleq_2| + |Aleq_3| ...
  remain_brach  $\leftarrow$  remain_brach / fix_branch
  remain_actions  $\leftarrow$  upper_k_actions

  # deal with action elements num larger than top_k
  if remain_actions not empty:
    # num_k_1 + num_k = num(remain_actions)
    k_1_num  $\leftarrow$  argmaxnum_k_1((k + 1)num_k_1 · knum_k < remain_branch)

  top_k_1_indices  $\leftarrow$  random(choose k_1_num from num(remain_actions))
  for each actions in remain_actions:
    drone_id  $\leftarrow$  drone_actions_tuples(use actions get id)
    k  $\leftarrow$  top_k + 1 if drone_id in top_k_1_indices else k
    k_a_record.add(tuple(drone_id, k))

  choose_a_vector  $\leftarrow$  []
  for idx, actions, k with each drone:
    if k == num(actions)
      choose_a_vector.add(actions)
    else:
      if random[0, 1) <  $\epsilon$ : # exploration epsilon
        choose_a_vector.add(random(choose k from all actions))
      else:
        choose_a_vector.add(random(actions[first kth a]))

  encode_action  $\leftarrow$  GENERATE_ENCODE_ACTIONS(choose_a_vector)

  # some action's element are not selected
  if not all (k == num(actions)) in each drone
    left_num  $\leftarrow$  (max_branch - len(encode_actions_set))
    rand_choose  $\leftarrow$  Random left_num action not in encode_action
    encode_action.add(rand_choose)

  return encode_action

```

```

FUNCTION HEURISTIC_SIMULATION_ACTION(state):
    action_vector ← HEURISTIC_ORDERED_VALID_ACTIONS(state)
    encode_action ← 0
    for each drone_id:
        dron_actions ← action_vector[drone_id]
        best_action ← action_vector[0]    #decrease ordered
        if random[0, 1) < ε:                #greedy epsilon for exploration
            next_action ← random( { dron_actions }/{ best_a })
        else:
            next_action ← best_action
        #encode the integer action for multi drones
        encode_action += next_action * |A|drone_id

    return encode_action

```

Class MyPlanner:

```

INITIALIZE:
    env, EXPLORATION_COE, MAX_SIMULATION_STEP, mcts_tree, last_action

```

```

FUNCTION PLAN(current_stae, planning_time_per_step):
    # Initialize tree
    child ← ROOT_CONTAIN_CHILDREN(last_action, current_state)
    if mcts_tree == None or child == None:
        mcts_tree ← new Tree                # new tree
    else:
        mcts_tree ← CUT_TREE_FROM_ROOT(child)    # cut tree

    t_start ← CURRENT_TIME()
    loop do:
        if CURRENT_TIME() - t_start >= planning_time_per_step:
            break loop

        # Selection
        select_node ← SELECTION()
        if select_node is done:                # terminate state
            BACK_PROPAGATION(select_node, expansion_reward)
            continue

        # Expansion
        expansion_node = EXPANSION(select_node)
        if expansion_node done:
            BACK_PROPAGATION(expansion_node, expansion_reward)
            continue

        # Simulation
        sim_reward ← SIMULATION(expansion_node)
        # Back_Propagation
        BACK_PROPAGATION(expansion_node, sim_reward)

        # finish MCST process, get best action
        best_action ← CHOOSE_BEST_ACTION()

        last_action ← best_action    # update

    return best_action

```

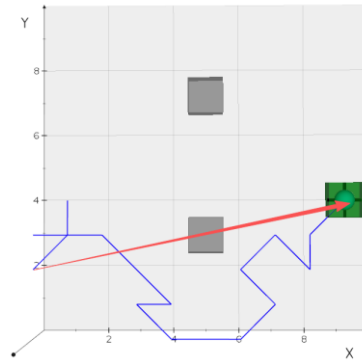
Analysis 1 – different drone number:

This test uses 4 configs environments with different number of drones (1, 2, 3, 4) that cannot change altitude.

Keep the obstacle number with 2 in a moderate difficulty.

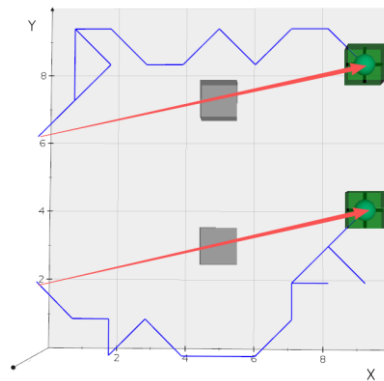
Config 1: 1 drone

Just one drone, and its direct path to the goal is blocked by one obstacle. The action space $|A| = 8$. Dimension is low.



Config 2: 2 drones

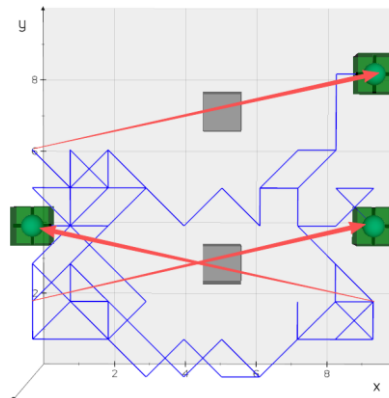
Similarly set second drone with the config 2, its path to the goal is blocked by the other obstacle. Action space $|A| = 8^2 = 64$, the dimension is still not very large.



Config 3: 3 drones

Keep the first two drones same with config 2. And place the third drone centrally symmetric with respect the first drone, using the first obstacle as the center of symmetry.

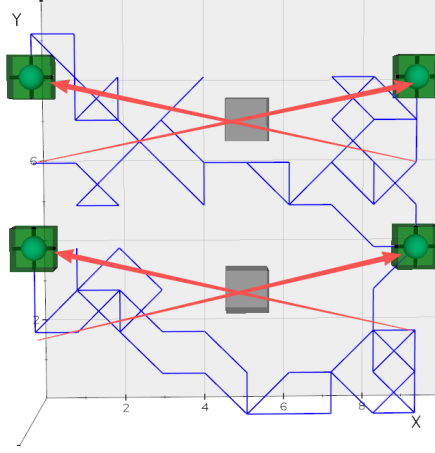
Action space $|A| = 8^3 = 512$. The dimension is a little large, need more computation load.



Config 4: 4 drones

Add the fourth drone similarly with third drone. It uses the second obstacle as the center of symmetry. And it is symmetric with second drone.

Action space $|A| = 8^4 = 4096$. This dimension is very huge, and it is hard to compute all actions for each state.



Implementation:

In this test, we change the drones' number to increase the action space. This test chooses the number from 1 to 4. Typically, this test only chooses two obstacles, because this number of obstacles is suitable. If there is one or no obstacle, every drone could easily reach the goal. Also, if there are many obstacles, even one drone can hardly reach the goal. So, two obstacles can provide certain difficulty and not influence the impact of action space. Also, obstacles are placed between all drones and their goals, preventing them from reaching the goal directly.

The config file: *config_q4_1.yaml* to *config_4_4.yaml*

The test script: *q4_test.py*

Planning time per step: 1s

For each config with different drones, we run 30 times for each test.

Statistic:

We collect the following statistic information:

Run time: Mean run time of each round where the drones terminate (Success or Collision).

Success rate: Percentage of how many rounds are success (all drones reach the goal).

Collision rate: Percentage of how many rounds that drones collide with obstacles or each other.

Steps: Mean steps that drones terminate; both successful cases and all round are recorded.

Reward: Mean total discounted reward that drones terminate; both successful and all round.

Test Result:

```
u7837228@l124lt30:~/Desktop/MultiDroneUnc$ python3 q4_test.py
[config_q4_1.yaml] with [drone num]: 1
Runtime: 14.11 ± 1.05 sec
Success rate: 93.3% ± 8.9%
Collision rate: 6.7% ± 8.9%
Total Steps: 14.1 ± 1.1
Success Steps: 13.8 ± 1.0
Total Reward: 57.6 ± 10.6
Success Reward: 65.3 ± 2.2

[config_q4_2.yaml] with [drone num]: 2
Runtime: 48.18 ± 9.73 sec
Success rate: 83.3% ± 13.3%
Collision rate: 10.0% ± 10.7%
Total Steps: 48.1 ± 9.7
Success Steps: 44.7 ± 9.4
Total Reward: 19.0 ± 24.0
Success Reward: 38.8 ± 21.5

[config_q4_3.yaml] with [drone num]: 3
Runtime: 28.73 ± 7.84 sec
Success rate: 73.3% ± 15.8%
Collision rate: 23.3% ± 15.1%
Total Steps: 28.7 ± 7.8
Success Steps: 30.2 ± 6.9
Total Reward: 71.1 ± 36.3
Success Reward: 123.5 ± 24.9

[config_q4_4.yaml] with [drone num]: 4
Runtime: 34.11 ± 8.45 sec
Success rate: 50.0% ± 17.9%
Collision rate: 43.3% ± 17.7%
Total Steps: 34.1 ± 8.4
Success Steps: 36.5 ± 7.9
Total Reward: 16.6 ± 46.1
Success Reward: 128.7 ± 40.9
```

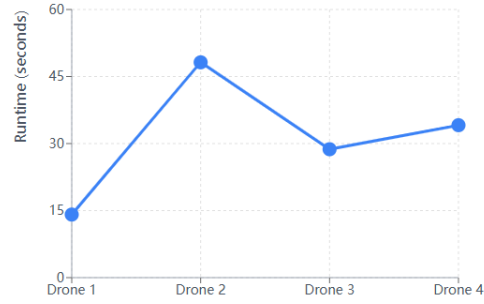
Drone num	1	2	3	4
Runtime(s)	14.11±1.05	48.18±9.73	28.73±7.84	34.11±8.45
Success rate(%)	93.3±8.9	83.3±13.3	73.3±15.8	50.0±17.9
Collision rate(%)	6.7±8.9	10.0±10.7	23.3±15.1	43.3±17.7
Total Steps	14.1±1.1	48.1±9.7	28.7±7.8	34.1±8.4
Success Steps	13.8±1.0	44.7±9.4	30.2±6.9	36.5±7.9
Total Reward	57.6±10.6	19.0±24.0	71.1±36.3	16.6±46.1
Success Reward	65.3±2.2	38.8±21.5	123.5±24.9	128.7±40.9

Discussion:

From this result, we can know that as the environment becomes more complex and the drones increases, the efficiency and accuracy of the planner decline. The increased action space further increases the overall system complexity, leading to higher planning difficulty, lower success rate and more steps, more collision occurrences.

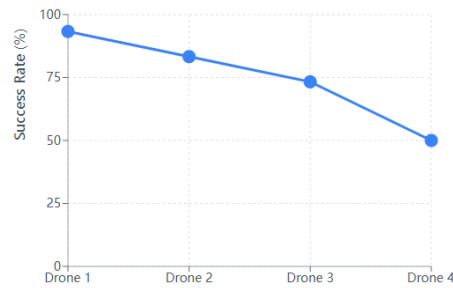
Runtime:

The case for 1 drone is shortest (14.11s). For 2 drones the runtime is highest, which may sampled unfavorable actions and led longer planning time. Overall, the results show an increasing trend. As the number of drones increases, the runtime grows and there is more time needed for termination.



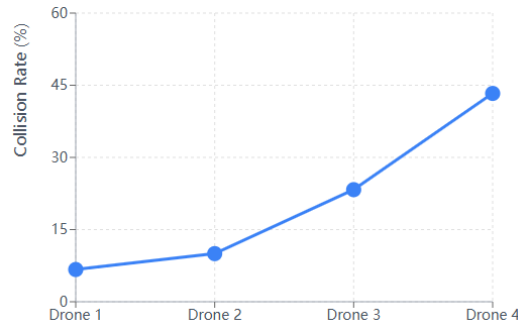
Success Rate:

The success rate for 1 drone is highest (93.3%) and the 4 drones is lowest (50.0%). So, it is hard to success for more drones. There is huge action space and need more steps to run and mor chance to meet collision.



Collision Rate:

It shows an opposite trend to the success rate. The reason is similar with success rate, as the number of drones increases, it is more likely occur collision. Also, all drones can only move in a plate, and cannot change altitude, the space is too narrow for many drones.



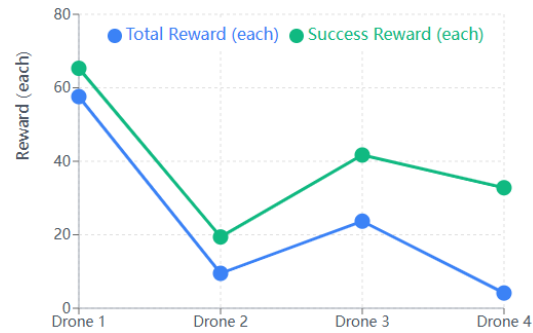
Steps:

For one drone case, the average total step is 14.1 ± 1.1 , it means the planner performs stably for the one-drone case. As the drone number increases, the success steps increase and become less stable. Then, success steps are slightly higher than total steps in (3 or 4 drones). It indicates more drones may collide easily and terminate the task at the short steps.



Reward:

Two drone case may sample worse and cause low reward. From other scenarios, the reward become less for when the drone number increase. The reward above is recorded for whole system. If we calculate them for per drone. We can know each drone will acquire lower average reward for multi-drone settings. And the variability of the reward also increases with the number of drones.



Drone num	1	2	3	4
Total Reward(each)	57.6	9.5	23.7	4.15
Success Reward(each)	65.3	19.4	41.7	32.8

Analysis 2 – different planning time per steps:

Config:

This test uses one of the configurations in (Analysis 1), with the number of drones set to three. It keeps a certain difficulty of action spaces. And this test will run under 5 different maximum planning time per steps, such as 0.1, 0.5, 1, 2, 5.

Implementation:

Use the configuration file in question 4: *config_q4_3.yaml*

The test script: *q5_test.py*

For each config with different drones, we run 30 times for each test.

Statistic:

Same with the question 4.

Run time, Success rate, Collision rate,

Steps (Success and Total), Reward (Success and Total)

Test Result:

```
u7837228@l124lt30:~/Desktop/MultiDroneUnc$ python3 q5_test.py
[Q5 test with [planning time]: 0.1
Runtime: 2.98 ± 0.64 sec
Success rate: 83.3% ± 13.3%
Collision rate: 16.7% ± 13.3%
Total Steps: 29.4 ± 6.4
Success Steps: 32.4 ± 7.1
Total Reward: 87.2 ± 29.9
Success Reward: 115.0 ± 22.5

[Q5 test with [planning time]: 0.5
Runtime: 15.57 ± 3.45 sec
Success rate: 76.7% ± 15.1%
Collision rate: 23.3% ± 15.1%
Total Steps: 31.1 ± 6.9
Success Steps: 33.9 ± 7.8
Total Reward: 72.1 ± 33.6
Success Reward: 113.9 ± 25.2

[Q5 test with [planning time]: 1.0
Runtime: 41.71 ± 11.13 sec
Success rate: 70.0% ± 16.4%
Collision rate: 16.7% ± 13.3%
Total Steps: 41.7 ± 11.1
Success Steps: 37.1 ± 9.7
Total Reward: 57.1 ± 34.4
Success Reward: 108.1 ± 28.3

[Q5 test with [planning time]: 2.0
Runtime: 48.63 ± 16.30 sec
Success rate: 56.7% ± 17.7%
Collision rate: 36.7% ± 17.2%
Total Steps: 24.3 ± 8.1
Success Steps: 22.4 ± 4.0
Total Reward: 54.3 ± 40.7
Success Reward: 149.5 ± 18.2

[Q5 test with [planning time]: 5.0
Runtime: 144.70 ± 42.60 sec
Success rate: 70.0% ± 16.4%
Collision rate: 23.3% ± 15.1%
Total Steps: 28.9 ± 8.5
Success Steps: 27.0 ± 6.0
Total Reward: 71.8 ± 36.9
Success Reward: 133.2 ± 20.9
```

Time per step(s)	0.1	0.5	1.0	2.0	5.0
Runtime(s)	2.98±0.64	15.57±3.45	41.71±11.13	48.63±16.30	144.70±42.6
Success rate(%)	83.3±13.3	76.7±15.1	70.0±16.4	56.7±17.7	70.0±16.4
Collision rate(%)	16.7±13.3	23.3±15.1	16.7±13.3	36.7±17.2	23.3±15.1
Total Steps	29.4±6.4	31.1±6.9	41.7±11.1	24.3±8.1	28.9±8.5
Success Steps	32.4±7.1	33.9±7.8	37.1±9.7	22.4±8.1	27.0±6.0
Total Reward	87.2±29.9	72.1±33.6	57.1±34.4	54.3±40.7	71.8±36.9
Success Reward	115.0±22.5	113.9±25.2	108.1±28.3	149.5±18.2	133.2±20.9

Discussion:

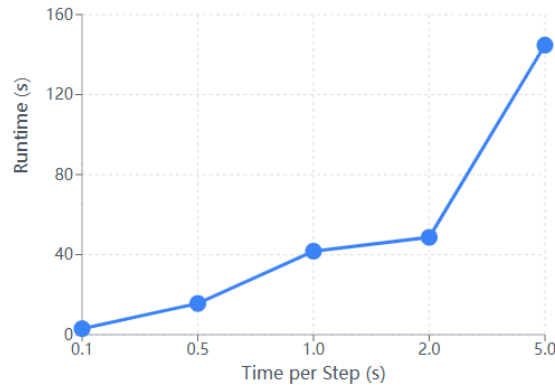
This result indicates the clear record for runtime. As the planning time per step increases, the runtime generally increases and the planner computes for each step. Success rate and reward tend to decrease then increase. And the steps are opposite. The collision rate has no obvious trend because all test case uses the same environment of drones and obstacles.

We can infer the heuristic method plays a dominant role in planning in shorter planning time per step. The planner cannot explore extensively and relies heavily on the heuristic method to guide the action decision. As planning time increases, exploration gradually influences the final decision. If the time is insufficient, the planner's decision may be worse than heuristic. So, the result shows the initial decrease in success rate and reward and later increase. At time 5 second, the planning time per step might be enough than 1 second and get better result.

As a result, if the planner has worse heuristic or chooses actions entirely at random, the performance will gradually become better as the planning time increase.

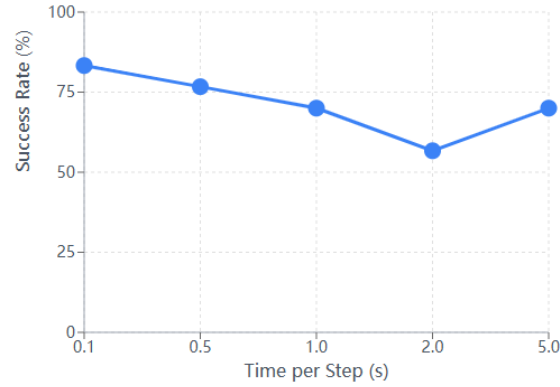
Runtime:

Runtime increases with planning time per step. The planner completes the task quickly with low variability. Also, the variability is larger for long planning time per step. The planner did more extensive exploration, and influence the change.



Success Rate:

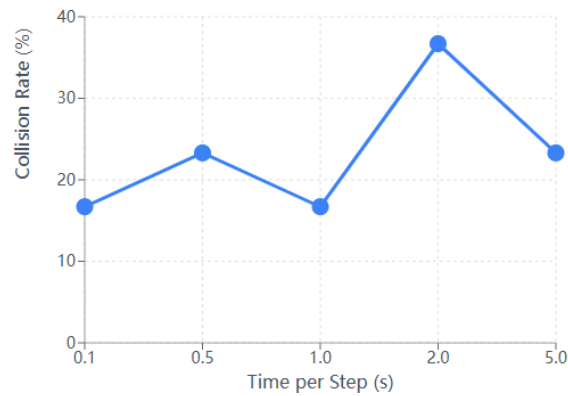
The success rate generally decreases as the planning time per step increases from 0.1 to 2.0 second. Then it increases at 5.0 second. We can know the heuristic guides the action decision with high success rate when planning time is short. At intermediate planning time, limited exploration may influence the heuristic result and cause suboptimal decisions. Although it becomes lower, but it is still better than random choice. Finally, at a planning time of 5 seconds per step, the planner identifies higher-quality actions and produces better overall planning results after sufficient exploration.



Collision Rate:

Since each test case just changes the planning time per step. Collision rate does not show the clear trend. The environment is same and the change is caused by randomly sample.

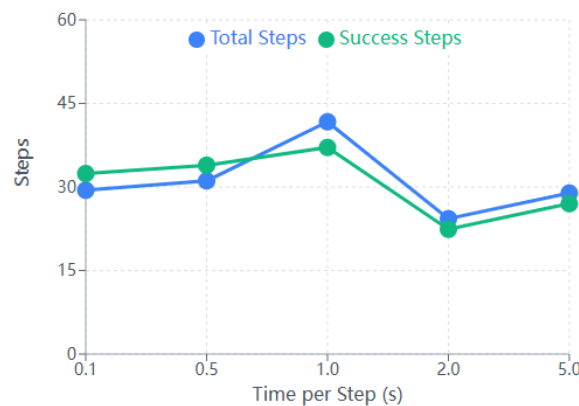
But for the variability, it increases at first then decrease. The reason is same about the heuristic and enough exploration.



Steps:

At short planning time, the steps are low, in the mediate, the steps are high, and in a long planning time, the steps return back to low. It shows that long planning time, the planner can find more optimal action. For the variability, has the same trend.

Comparing total steps and success steps, total steps are lower than success step at short planning time. This indicates that planner makes decision mainly by heuristic, and will cause more collisions and early termination. For the long planning time, the planner can explore more and avoid collisions (penalty with low rewards). In this case, it terminates by the max steps limitation. So, the total steps are higher than success steps.



Reward:

Since all test cases use the same environment, we do not need to calculate the average reward for each drone. Similar to the previous analysis, short planning time yields high rewards, which then decrease and increase. The reason for this trend is the same as discussed earlier (heuristic and exploration).



Question 6:

This method adopts MCTS to make the multi-drone planner. Since the action space becomes large as the large, we cannot explore every action. I combine the heuristic and ϵ -greedy to choose which actions that need to explore. 20% probability for exploration and 80% for heuristic.

Although these methods can let the planner solve the problem more quickly and accurately. There are still several limitations.

1. High-dimensional limitation:

Each drone has 8 or 26 actions and there are $|A|^N$ actions for the combined action space. As a result, MCTS cannot expand every action from a certain state. Eg, 17576 actions (3 drones 26 actions each). So, we can only select a subset of actions from it to keep the MCTS branch is small. And, we cannot explore all cases, and this may cause some optimal actions to be missed.

2. Reuse Tree Limitation:

This planner expands the node through the simulator. If the actual state after the action differs from the simulated state, the existing tree will become invalid. Building a tree need significant computation and the tree contains valuable statistic information. Discarding the tree is highly wasteful. But this form of MCTS is hard to reuse the tree only in certain cases.

3. Computation Limitation:

After testing the time cost (**3 drones**) of heuristic and simulator time for each rollout step.

Heuristic action selection(mean): **0.0002619s**

Simulator to get next state(mean): **0.0005345s**

Total in rollout(mean): **0.0007993s**

If we need run 12 steps in each simulation step, we need **0.009592 seconds**.

So, we can only expand $1/0.009592 \approx 104$ nodes. This number is extremely small compared with the size action space.

As a result, the computation is very heavy, and it is unrealistic to expand all possible actions during planning. We need some strategy to choose some actions that might be better.

4. Randomness and stableness:

Through the test result, we can know the result changes a lot even in the same test environment. If the planner samples some worse actions, and explores some bad positions, the runtime, success rate and reward will become worse. As a result, we can hardly give the deterministic result in a certain state.

5. Parameter Sensitivity

There are many tunable parameters that affect the performance of the planner significantly.

Ucb coefficient, Max simulation steps, Greedy epsilon,

Goal distance weight, Obstacle distance weight

It is hard to set the optimal configuration values across each parameter. Small changes may lead to performance differences. And the same configuration may perform different across different environment. Some adaptive method might mitigate this limitation.